PROJECT-DOCS

Developer Guide

project-docs skill

VERSION 1.0.1

DATE 2026-10-18

AUTHOR project-docs maintainers

Contents

Prerequisites 3

Clone to running 3

Project structure 4

Common workflows 4

- Scan this repository without its samples 4
- Add a document type 4

Testing 5

Debugging tips 5

Related documents 5

Assumptions and open questions 5

project-docs is a Claude Code skill: a `SKILL.md` workflow plus Python 3 scripts that turn a repository into branded PDF documentation. This guide covers setting up, running and testing it. For the design rationale, see `DEV_GUIDE.md` at the repo root.

Prerequisites

Tool	Version	Source
Python	3.9 or newer (developed on 3.9.6)	`tomli; python_version < "3.11"` in `requirements.txt`
A PDF renderer	WeasyPrint + Pango, **or** Chromium/Chrome	`requirements.txt`, `build_pdf.py`
mermaid-cli (`mmdc`)	optional	`render_mermaid()` in `scripts/build_pdf.py`
Git	optional; enables git facts in scans	`git_info()` in `scripts/scan_project.py`

> **NOTE**
> On the machine that generated this guide, WeasyPrint couldn't load Pango, so every PDF was rendered by the installed Google Chrome through Playwright. The fallback is automatic.

Clone to running

1. Clone the repository:

```
git clone https://github.com/zalhashemi/proj-docs.git
cd proj-docs
```

2. Create the environment. This makes a venv in `~/.cache/project-docs/venv` and installs `skills/project-docs/requirements.txt`:

```
python3 skills/project-docs/scripts/bootstrap.py --install
```

Add `--with-chromium` if you have neither Pango nor Google Chrome. Set `PROJECT_DOCS_PYTHON` to use an existing interpreter, or `PROJECT_DOCS_VENV` to put the venv elsewhere (`scripts/bootstrap.py`).

3. Check that everything works:

```
~/.cache/project-docs/venv/bin/python -m pytest tests -q
```

Expected: every test passes. Rendering tests skip themselves when no renderer is installed.

Project structure

```
proj-docs/
├── .claude-plugin/        # plugin.json + marketplace.json
├── skills/project-docs/   # the skill users install
│   ├── SKILL.md           # trigger description + workflow (≤150 lines)
│   ├── catalog.yaml       # 37 document types, presets, scan_keys
│   ├── reference/  templates/  themes/  assets/
│   └── scripts/           # bootstrap.py + 8 tools, shared helpers in _common.py
├── tests/                 # pytest suites, fixtures/, sample_docs/
├── samples/               # rendered example projects
└── docs/                  # this guide (the skill run on its own repo)
```

Common workflows

Run these from the repository root with `PY=~/.cache/project-docs/venv/bin/python` and `S=skills/project-docs/scripts`.

Command	What it does	Tested
`$PY $S/scan_project.py .`	Writes `docs/.project-docs/scan.json`	✓
`$PY $S/detect_theme.py . --use midnight`	Writes the theme used for this guide	✓
`$PY $S/build_pdf.py docs/dev-guide.md`	Renders `docs/pdf/dev-guide.pdf`	✓
`$PY $S/verify_pdf.py docs/pdf/*.pdf`	Runs checks and writes thumbnails	✓
`$PY $S/state.py diff`	Lists docs affected by code changes	✓
`$PY $S/preview_themes.py`	Renders one swatch page per theme	✓

Every script answers `--help`. For example:

```
$ $PY $S/build_pdf.py --help
usage: build_pdf.py [-h] [--theme THEME] [--out OUT]
```

Scan this repository without its samples

The repo contains fake projects for testing. Exclude them, or their routes and colours mix in:

```
$PY $S/scan_project.py . --exclude 'samples/*' \
  --exclude 'tests/fixtures/*' --exclude 'tests/sample_docs/*'
```

Add a document type

1. Add an entry to `catalog.yaml`.
2. Add a matching template in `templates/`.
3. Run `pytest tests/test_catalog.py`.

The README section "Add a document type" has a full example.

Testing

```
~/.cache/project-docs/venv/bin/python -m pytest tests -q
```

Suite	Covers
`test_catalog.py`	Catalog schema, templates, presets, theme contrast, SKILL.md size, plugin manifests
`test_scan.py`	Scanner on 3 fixtures; empty, unknown, gitignored and git-repo cases
`test_theme.py`	Detection for Tailwind v3/v4, shadcn, MUI, Flutter, Android/iOS, dark-only, logo-only
`test_render.py`	Markdown pipeline, renderer fallback, PDF output, verify checks
`test_finance_state.py`	Forecast and budget maths, update tracking, hand-edit detection
`test_e2e.py`	Full workflow on each fixture with drafted docs

Rendering tests skip themselves when no renderer is installed.

Debugging tips

- Read the `warnings` array in `build_pdf.py`'s JSON output first. It reports renderer fallback, Mermaid diagrams shown as code, unfilled `{{placeholders}}` and a missing logo.
- Look at pages, not just test results: `verify_pdf.py --dpi 80` writes PNGs to `docs/.project-docs/preview/`.
- Verify reports text "outside the margins" when a table is too wide. Use fewer columns or wrap long tokens in backticks.
- If imports fail, run `bootstrap.py`: it reports which packages are missing and which Python it found.

Related documents

Architecture · Testing Strategy · Contributing

Assumptions and open questions

#	Item	Type	Why unverified	Who can confirm
A1	`bootstrap.py --with-chromium` works on other machines	assumption	Chrome was used locally; CI downloads Chromium	maintainer